

# Building and Validating an AWS Monitoring and Security Incident Response Workflow

This project was intentionally designed to go beyond simply enabling AWS services or following documentation. The objective was to build a complete monitoring and security detection workflow, deliberately trigger real operational and security issues, and confirm that each AWS service responded correctly under real conditions.

Instead of using sample events or test findings, every alert and security event in this project was generated through live EC2 instances and actual system activity. This approach ensured that detection, alerting, automation, and logging were all validated end to end.

To do this safely and realistically, I created a small AWS environment with clearly separated development and production systems and used controlled tools to simulate both performance failures and security threats.

## Initial Environment Setup

Two EC2 instances running Amazon Linux 2023 were launched to represent separate environments. A Dev instance was used to generate load and security testing activity, while a Prod instance was monitored and protected.

- 1 Dev instance: used to generate load and security testing activity
- 2 Prod instance: used as the monitored and protected system

Each instance was tagged with environment metadata to clearly identify its role and simplify troubleshooting. Separating the environments allowed disruptive activity to be generated safely from Dev without interfering with production monitoring logic.

After launching the instances, connectivity was verified via SSH and basic networking and access controls were validated.

## Testing Tooling

- 1 Installed the stress utility on the Dev instance to generate sustained CPU load
- 2 Used falldate on the Prod instance to simulate disk space exhaustion

These tools were selected because they create realistic system conditions while remaining easy to clean up after testing, preventing permanent damage or unintended downtime.

## Extending Monitoring with the CloudWatch Agent

By default, EC2 only exposes basic metrics such as CPU utilization. Since disk usage and memory were critical indicators, the CloudWatch Agent was installed on both instances.

After installation, disk and memory metrics initially failed to appear in CloudWatch. Reviewing the agent logs revealed authentication errors indicating the agent could not send metrics to AWS.

- 1 Created a dedicated IAM role with CloudWatchAgentServerPolicy
- 2 Attached the role to both EC2 instances
- 3 Restarted the CloudWatch Agent service

Once the role was attached and the agent restarted, custom metrics appeared under the CWAgent namespace, confirming metrics were collected directly from the operating system.

## Creating CloudWatch Alarms

- 1 High CPU alarm for the Dev instance (85% threshold)
- 2 Low Disk Space alarm for the Prod instance (80% threshold)

Each alarm was configured to publish notifications to an SNS topic and trigger a Lambda function to automate responses. SNS email subscriptions were confirmed before testing.

## Testing Performance Monitoring in Practice

CPU stress tests on the Dev instance successfully triggered the High CPU alarm. Disk space exhaustion on the Prod instance triggered the Low Disk Space alarm.

- 1 Verified alarm state changes in CloudWatch
- 2 Confirmed SNS email delivery
- 3 Validated Lambda execution via CloudWatch logs

After testing, cleanup actions were performed to ensure alarms returned to the OK state and systems recovered as expected.

## Automating Responses with Lambda

The Lambda function processed SNS-delivered alarm notifications and identified affected EC2 instances. Native EC2 metrics and CloudWatch Agent metrics required different parsing logic.

- 1 Extracted identifiers from alarm dimensions

- 2 Translated hostnames into EC2 instance IDs when required
- 3 Applied Issue tags (HighCPU or LowDisk) to affected instances

Lambda execution logs and EC2 tags confirmed that automation was operating on real infrastructure.

## Enabling GuardDuty for Security Detection

AWS GuardDuty was enabled to introduce a security detection layer. To validate detection, an aggressive nmap scan was executed from the Dev instance targeting the Prod instance.

After GuardDuty's analysis window, a finding appeared identifying reconnaissance activity against the Prod instance.

- 1 Source IP: Dev instance
- 2 Target resource: Prod instance
- 3 Detection type: Recon:EC2/PortProbeUnprotectedPort

## Connecting GuardDuty to Automated Response

An EventBridge rule was created to route GuardDuty findings to the Lambda function. Initial issues were resolved by validating event patterns, region alignment, and permissions.

Once corrected, EventBridge successfully invoked Lambda, confirming the security automation pipeline.

## Incident Review and Validation

- 1 Reviewed GuardDuty findings
- 2 Confirmed EventBridge rule execution
- 3 Reviewed Lambda execution logs
- 4 Verified EC2 Issue tags

System and SSH logs on the Prod instance were reviewed to confirm no successful exploitation occurred.

## Final Outcome and Lessons Learned

- 1 GuardDuty detections are intentionally delayed
- 2 IAM roles and region consistency are common failure points

- 3 Logs are often the only proof automation executed
- 4 Monitoring systems must be validated through real testing

By the end of the project, the environment could detect performance issues, identify security threats, and automatically respond using AWS-native services, closely reflecting real-world cloud operations.